

SAMURAI Robot

Теория автоматического управления

ОДУ-модель, синтез регуляторов, MPC, наблюдатель состояния

Технический справочник по регулятору движения робота

Объект управления:	2-моторный гусеничный робот (Adept HAT)
Размерность состояния:	$n = 5$ (координаты + скорости)
Размерность управления:	$r = 2$ (linear, angular)
Период дискретизации:	$T_s = 0.05$ с (20 Гц)
Метод линеаризации:	ряд Тейлора в опорной точке
Синтез регуляторов:	модальный, ЛКР, MPC
Наблюдатель:	полноразмерный (Luenberger)
Реализация:	MATLAB (offline) $\rightarrow$ YAML $\rightarrow$ Python (онлайн)

Источник теории: Козлов В. Н., Куприянов В. Е., Шашихин В. Н.
“Теория автоматического управления”, СПбГПУ, 2008

April 27, 2026

Contents

Введение	3
1 Математическая модель робота SAMURAI	5
1.1 Физическая постановка	5
1.2 Нелинейная ОДУ модели	5
1.3 Линеаризация в опорной точке	6
1.3.1 Вычисление матриц Якоби	7
1.3.2 Численные значения	7
1.4 Анализ управляемости	7
1.5 Дискретизация	8
1.5.1 Соответствие собственных значений	8
2 Синтез регуляторов: модальный и оптимальный	9
2.1 Постановка задачи стабилизации	9
2.2 Модальный регулятор (pole placement)	9
2.2.1 Идея метода	9
2.2.2 Каноническая форма Фробениуса	10
2.2.3 Многомерный вход	10
2.3 Оптимальный регулятор (ЛКР)	10
2.3.1 Уравнение Риккати	11
2.3.2 Условия существования решения	11
2.3.3 Выбор весовых матриц	11
2.4 Регулирование с ненулевой уставкой	12
2.5 Наблюдатель состояния (Луенбергер)	12
2.5.1 Принцип разделения	13
2.5.2 Условие наблюдаемости	13
2.6 Численный пример для робота	13
3 Прогнозирующее управление (MPC)	15
3.1 Идея метода	15
3.2 Математическая постановка	15
3.3 Сведение к QR	16
3.3.1 Развёртка динамики	16
3.3.2 QR в стандартной форме	16
3.4 Алгоритм решения	16
3.4.1 Без ограничений: явное решение	17
3.4.2 С ограничениями: QR-солвер	17
3.4.3 Терминальный штраф $\mathbf{P}_f$	17

3.5	Параметры MPC для SAMURAI	17
3.6	Подбор поведения и «ошибочные» режимы	18
4	Реализация в коде робота	19
4.1	Архитектура	19
4.2	Конфигурация	19
4.3	Python-модуль управления	20
4.3.1	Загрузка матриц	21
4.3.2	Регулятор LQR	21
4.3.3	MPC: явное решение + проекция	22
4.4	Тестирование	22
4.4.1	test_state_space_model.py	22
4.4.2	test_lqr_controller.py	23
4.4.3	test_mpc_controller.py	23
4.5	Примеры использования	23
4.5.1	Запуск симуляции с MPC	23
4.5.2	Расчёт нового регулятора в MATLAB	24
4.5.3	«Ошибочный» режим для демонстрации	24
4.6	Заключение	24

Введение

Документ содержит полное математическое описание системы автоматического управления (САУ) гусеничным роботом SAMURAI. Изложение построено по схеме «**модели** → **анализ** → **синтез**» согласно учебному пособию *Козлов В. Н., Куприянов В. Е., Шашикин В. Н. “Теория автоматического управления”* (СПбГПУ, 2008).

Постановка задачи

Робот SAMURAI имеет 2 задних мотора (M1, M2) с дифференциальной кинематикой и должен отрабатывать заданные траектории движения. Существующая система управления использует прямую трансляцию команд скорости (`cmd_vel`) в ШИМ моторов через статические калибровочные коэффициенты, что:

- не учитывает инерционность моторов ($\tau \approx 0.15$ с);
- не оптимизирует управление по интегральному критерию;
- не предсказывает поведение системы на горизонте.

Цель работы — построить **линейную модель** робота, синтезировать **ЛКР** и **МРС-регулятор**, и встроить их в код управления так, чтобы параметры регулятора были **редактируемы из конфига**. Это позволяет:

1. улучшать поведение робота, подбирая матрицы **Q**, **R**;
2. **намеренно ухудшать** поведение для отладки и демонстрации неустойчивости (выбор «неправильных» полюсов и весов);
3. менять режим управления на лету (off/LQR/MPC) без перекомпиляции.

Структура документа

Глава	Содержание
1. Модель робота	Нелинейная ОДУ, линеаризация, дискретизация
2. Синтез регуляторов	Модальный, ЛКР через Риккати
3. МРС	Прогнозирующее управление, QP-задача
4. Реализация в коде	MATLAB → YAML → Python

Используемые обозначения

$\mathbf{x}(t) \in \mathbb{R}^n$	вектор состояния объекта
$\mathbf{u}(t) \in \mathbb{R}^r$	вектор управляющих воздействий
$\mathbf{y}(t) \in \mathbb{R}^m$	вектор наблюдаемых выходов
$\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$	матрицы модели в пространстве состояний
$\mathbf{K}$	матрица обратной связи (регулятор)
$\mathbf{L}$	матрица обратной связи наблюдателя
$\mathbf{Q}, \mathbf{R}$	весовые матрицы критерия качества
$\mathbf{P}$	решение уравнения Риккати
T_s	период дискретизации
N	горизонт прогноза МРС

Chapter 1

Математическая модель робота SAMURAI

1.1 Физическая постановка

Робот SAMURAI — гусеничное шасси с дифференциальным приводом. Контроллер PCA9685 управляет двумя задними моторами (M1 левый, M2 правый); передние моторы физически отсутствуют (см. `hardware_presets/samurai_default.yaml`).

Параметры объекта (`config.yaml`)

```
simulator:
  wheel_base: 0.17 # m (track separation)
  max_linear: 0.30 # m/s
  max_angular: 2.00 # rad/s
```

Команда движения формируется в виде вектора $\mathbf{u} = (u_v, u_\omega)^\top$, где u_v — желаемая линейная скорость [м/с], u_ω — желаемая угловая скорость [рад/с]. Дифференциальная кинематика преобразует $\mathbf{u}$ в скорости гусениц:

$$v_L = u_v + \frac{B}{2} u_\omega, \quad v_R = u_v - \frac{B}{2} u_\omega, \quad (1.1)$$

где B — ширина базы (`wheel_base`).

1.2 Нелинейная ОДУ модели

В отличие от тривиальной кинематической модели, реальный робот имеет **инерционность моторов**: команда u_v не отрабатывается мгновенно, а через апериодическое звено первого порядка с постоянной времени τ_v . Аналогично для угловой скорости (постоянная τ_ω).

Введём **вектор состояния** ($n = 5$):

$$\mathbf{x}(t) = (p_x, p_y, \theta, v, \omega)^\top \quad (1.2)$$

- p_x, p_y — координаты центра робота в мировой системе [м];

- θ — угол курса (yaw) [рад];
- v — фактическая линейная скорость [м/с];
- ω — фактическая угловая скорость [рад/с].

Нелинейная модель в форме $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$

$$\begin{cases} \dot{p}_x = v \cos \theta, \\ \dot{p}_y = v \sin \theta, \\ \dot{\theta} = \omega, \\ \dot{v} = \frac{1}{\tau_v} (u_v - v), \\ \dot{\omega} = \frac{1}{\tau_\omega} (u_\omega - \omega). \end{cases} \quad (1.3)$$

Первые три уравнения — кинематика дифференциального робота (идентична уравнениям 1.1 из учебника Козлова, формулы для $\dot{\mathbf{x}}$ и $\dot{\mathbf{z}}$). Последние два — динамика моторов как RC-звено с постоянной времени τ , определяемой механической инерцией ротора и противо-ЭДС.

Примечание

Постоянные времени $\tau_v \approx 0,15$ с, $\tau_\omega \approx 0,10$ с оценены экспериментально по графику разгона (см. `tests/test_motor_lag.py`). Для других шасси значения нужно переопределить — они вынесены в `config.yaml` раздел `control.plant`.

1.3 Линеаризация в опорной точке

Выберем **опорный режим** — равномерное прямолинейное движение вперёд со скоростью v_0 :

$$\bar{\mathbf{x}} = (\bar{p}_x, \bar{p}_y, 0, v_0, 0)^\top, \quad \bar{\mathbf{u}} = (v_0, 0)^\top. \quad (1.4)$$

Подстановка $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ в (1.3) даёт правые части $\dot{\bar{\mathbf{x}}} = (v_0, 0, 0, 0, 0)^\top$, что соответствует нулевым отклонениям состояния от опорной траектории при постоянном v_0 .

Согласно методу линеаризации (см. учебник п. 1.1, формулы 1.2), уравнение для отклонений $\Delta \mathbf{x} = \mathbf{x} - \bar{\mathbf{x}}$, $\Delta \mathbf{u} = \mathbf{u} - \bar{\mathbf{u}}$ имеет вид:

$$\Delta \dot{\mathbf{x}}(t) = \mathbf{A} \Delta \mathbf{x}(t) + \mathbf{B} \Delta \mathbf{u}(t), \quad \mathbf{A} = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}, \quad \mathbf{B} = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}. \quad (1.5)$$

1.3.1 Вычисление матриц Якоби

Распишем частные производные функций f_i по x_j :

$$\begin{aligned}\frac{\partial f_1}{\partial \theta} &= -v \sin \theta|_0 = 0, & \frac{\partial f_1}{\partial v} &= \cos \theta|_0 = 1, \\ \frac{\partial f_2}{\partial \theta} &= v \cos \theta|_0 = v_0, & \frac{\partial f_2}{\partial v} &= \sin \theta|_0 = 0, \\ \frac{\partial f_3}{\partial \omega} &= 1, & \frac{\partial f_4}{\partial v} &= -1/\tau_v, \\ \frac{\partial f_5}{\partial \omega} &= -1/\tau_\omega.\end{aligned}$$

Остальные элементы матрицы $\mathbf{A}$ равны нулю. Получаем:

Линеаризованная модель в пространстве состояний

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & v_0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1/\tau_v & 0 \\ 0 & 0 & 0 & 0 & -1/\tau_\omega \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/\tau_v & 0 \\ 0 & 1/\tau_\omega \end{pmatrix}. \quad (1.6)$$

Матрица наблюдения $\mathbf{C}$ выбирается по доступным датчикам. В роботе SAMURAI прямо измеряются p_x, p_y (одометрия), θ (IMU), а v, ω восстанавливаются из последовательных позиций. Будем считать наблюдаемыми все 5 координат:

$$\mathbf{C} = \mathbf{I}_5, \quad \mathbf{D} = \mathbf{0}_{5 \times 2}. \quad (1.7)$$

1.3.2 Численные значения

При $v_0 = 0,2$ м/с (профиль normal), $\tau_v = 0,15$ с, $\tau_\omega = 0,10$ с:

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0.2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -6.667 & 0 \\ 0 & 0 & 0 & 0 & -10 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 6.667 & 0 \\ 0 & 10 \end{pmatrix}. \quad (1.8)$$

1.4 Анализ управляемости

Объект полностью управляем тогда и только тогда, когда матрица управляемости имеет полный ранг (учебник, формула перед п. 3.1):

$$\mathbf{S}_y = [\mathbf{B} \mid \mathbf{AB} \mid \mathbf{A}^2\mathbf{B} \mid \dots \mid \mathbf{A}^{n-1}\mathbf{B}], \quad \text{rank } \mathbf{S}_y = n. \quad (1.9)$$

Для нашей системы $n = 5$, $r = 2$, поэтому $\mathbf{S}_y$ имеет размер 5×10 . Вычисление в MATLAB (matlab/analyze_system.m, см. главу 4) показывает $\text{rank } \mathbf{S}_y = 5$ при $v_0 \neq 0$ — объект полностью управляем.

Внимание

При $v_0 = 0$ опорная точка вырождена: координата p_y становится неуправляемой (нельзя двигаться вбок без вращения). Для синтеза регулятора необходимо $v_0 > 0$ или модель в окрестности нетривиальной траектории.

1.5 Дискретизация

Для реализации регулятора в реальном времени переходим к дискретной модели с периодом $T_s = 0,05$ с (20 Гц — частота цикла `motor_node`). Согласно п. 1.5 учебника, дискретизация ЗОН (zero-order hold) даёт:

$$\mathbf{A}_d = e^{\mathbf{A}T_s}, \quad \mathbf{B}_d = \int_0^{T_s} e^{\mathbf{A}\tau} d\tau \cdot \mathbf{B}, \quad (1.10)$$

что эквивалентно вычислению блочной матричной экспоненты:

$$\exp\left(\begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} T_s\right) = \begin{pmatrix} \mathbf{A}_d & \mathbf{B}_d \\ \mathbf{0} & \mathbf{I} \end{pmatrix}. \quad (1.11)$$

Дискретная модель:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k, \quad \mathbf{y}_k = \mathbf{C} \mathbf{x}_k. \quad (1.12)$$

1.5.1 Соответствие собственных значений

Собственные значения непрерывной и дискретной систем связаны соотношением (п. 3.1 учебника):

$$\tilde{\lambda}_i = e^{\lambda_i T_s}. \quad (1.13)$$

Условия устойчивости:

- непрерывная: $\operatorname{Re} \lambda_i < 0$ — собственные значения в левой полуплоскости;
- дискретная: $|\tilde{\lambda}_i| < 1$ — внутри единичного круга.

Примечание

Для непрерывного робота $\mathbf{A}$ имеет три нулевых собственных значения (апериодические интеграторы по p_x, p_y, θ) и два отрицательных: $-1/\tau_v, -1/\tau_\omega$. Объект **не устойчив асимптотически** (но и не неустойчив — это интегратор), что и ожидаемо: робот без управления продолжает двигаться по инерции.

Chapter 2

Синтез регуляторов: модальный и оптимальный

В этой главе строится статический регулятор $\mathbf{u} = -\mathbf{K} \mathbf{x}$, размещающий полюса замкнутой системы в желаемых точках. Излагаются два подхода (по разделам 3.1 и 3.3 учебника): **модальный** (выбор полюсов вручную) и **оптимальный** ЛКР (минимизация интегрального критерия).

2.1 Постановка задачи стабилизации

Дана линейная стационарная модель (по результатам главы 1):

$$\dot{\mathbf{x}}(t) = \mathbf{A} \mathbf{x}(t) + \mathbf{B} \mathbf{u}(t), \quad \mathbf{x}(0) = \mathbf{x}^0. \quad (2.1)$$

Требуется построить регулятор статической обратной связи:

$$\boxed{\mathbf{u}(t) = -\mathbf{K} \mathbf{x}(t)}, \quad (2.2)$$

такой, что замкнутая система $\dot{\mathbf{x}} = (\mathbf{A} - \mathbf{BK})\mathbf{x}$ асимптотически устойчива, и переходный процесс удовлетворяет требованиям по длительности и перерегулированию.

2.2 Модальный регулятор (pole placement)

2.2.1 Идея метода

Зададим желаемые собственные значения замкнутой системы $\{\lambda_i^{\text{уст}}\}_{i=1}^n$ и подберём $\mathbf{K}$ так, чтобы спектр $\sigma(\mathbf{A} - \mathbf{BK})$ совпал с этим множеством.

Согласно п. 3.1.3 учебника, для гарантии экспоненциального переходного процесса заданной длительности t_n выбираются вещественные собственные значения, удовлетворяющие

$$\min_i |\lambda_i^{\text{уст}}| \geq \frac{t_n}{3}. \quad (2.3)$$

2.2.2 Каноническая форма Фробениуса

Для одномерного входа ($r = 1$) модальный регулятор синтезируется через приведение к канонической форме (п. 3.1.2 учебника):

$$\mathbf{A}_f = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ -a_n & -a_{n-1} & \cdots & & -a_1 \end{pmatrix}, \quad \mathbf{b}_f = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}. \quad (2.4)$$

Преобразование подобия $\mathbf{Q} = \mathbf{S}_y \mathbf{T}$, где

$$\mathbf{T} = \begin{pmatrix} a_{n-1} & a_{n-2} & \cdots & a_1 & 1 \\ a_{n-2} & a_{n-3} & \cdots & 1 & 0 \\ \vdots & & & & \vdots \\ 1 & 0 & \cdots & 0 & 0 \end{pmatrix}, \quad (2.5)$$

a_i — коэффициенты характеристического полинома исходной матрицы $\chi_n(\lambda) = \det(\lambda \mathbf{I} - \mathbf{A}) = \lambda^n + a_1 \lambda^{n-1} + \dots + a_n$.

Параметры регулятора в исходном базисе (формула 3.4 учебника):

$$\mathbf{K} = (\mathbf{Q}^\top)^{-1}(\mathbf{a} - \mathbf{a}_{\text{уст}}), \quad (2.6)$$

где $\mathbf{a}_{\text{уст}} = (a_n^{\text{уст}}, a_{n-1}^{\text{уст}}, \dots, a_1^{\text{уст}})^\top$ — коэффициенты желаемого характеристического полинома $\prod_{i=1}^n (\lambda - \lambda_i^{\text{уст}})$.

2.2.3 Многомерный вход

При $r > 1$ (как у нашего робота: u_v и u_ω) канонизация по (2.4) напрямую неприменима. На практике используют функцию `place(A,B,p)` пакета Control System Toolbox, реализующую алгоритм Tits–Yang (численно устойчивый). Для нашей задачи в MATLAB:

matlab/design_modal.m

```
1 poles_des = [-2; -3; -4; -8; -10]; % желаемые полюса
2 K_modal   = place(Ad, Bd, exp(poles_des * Ts)); % дискретный
```

2.3 Оптимальный регулятор (ЛКР)

Подход модального синтеза требует ручного выбора полюсов, что не масштабируется при многомерном состоянии. **Линейно-квадратичный регулятор** (ЛКР) автоматически минимизирует интегральный критерий качества (п. 3.3 учебника):

Критерий качества

$$J = \int_0^\infty (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{u}^\top \mathbf{R} \mathbf{u}) dt, \quad \mathbf{Q} = \mathbf{Q}^\top \succeq 0, \quad \mathbf{R} = \mathbf{R}^\top \succ 0. \quad (2.7)$$

Здесь $\mathbf{Q}$ штрафует отклонение состояния (точность), $\mathbf{R}$ — расход управления (энергию). Подбор соотношения $\mathbf{Q}/\mathbf{R}$ определяет компромисс «точность $\leftrightarrow$ затраты».

2.3.1 Уравнение Риккати

Оптимальное управление (формула 3.10 учебника) имеет вид

$$\mathbf{u}^*(t) = -\mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P} \mathbf{x}(t) = -\mathbf{K} \mathbf{x}(t), \quad (2.8)$$

где $\mathbf{P}$ — симметричная положительно определённая матрица, удовлетворяющая алгебраическому уравнению Риккати (CARE):

Непрерывное уравнение Риккати

$$\mathbf{A}^\top \mathbf{P} + \mathbf{P} \mathbf{A} - \mathbf{P} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^\top \mathbf{P} = -\mathbf{Q}. \quad (2.9)$$

Дискретный аналог (DARE):

$$\mathbf{A}^\top \mathbf{P} \mathbf{A} - \mathbf{P} - \mathbf{A}^\top \mathbf{P} \mathbf{B} \bar{\mathbf{R}}^{-1} \mathbf{B}^\top \mathbf{P} \mathbf{A} = -\mathbf{Q}, \quad \bar{\mathbf{R}} = \mathbf{R} + \mathbf{B}^\top \mathbf{P} \mathbf{B}, \quad (2.10)$$

матрица регулятора $\mathbf{K}_d = \bar{\mathbf{R}}^{-1} \mathbf{B}^\top \mathbf{P} \mathbf{A}_d$.

2.3.2 Условия существования решения

По п. 3.3 учебника, единственное положительно определённое $\mathbf{P}$ существует при выполнении трёх условий:

1. Объект $(\mathbf{A}, \mathbf{B})$ управляем ($\text{rank } \mathbf{S}_y = n$);
2. $\mathbf{R} \succ 0$;
3. $\mathbf{Q} \succ 0$, либо $\mathbf{Q} = \mathbf{H}^\top \mathbf{H}$ при наблюдаемой паре $(\mathbf{A}^\top, \mathbf{H}^\top)$.

Для робота все три условия выполнены при $v_0 > 0$.

2.3.3 Выбор весовых матриц

Согласно рекомендации [13] из учебника (п. 3.3 после формулы 3.12), естественный выбор:

$$\mathbf{R} = \mathbf{I}_r, \quad \tilde{\mathbf{Q}} = \text{diag}\left(\frac{1}{(y^{ct})^2}, \frac{1}{(t_{\max}/3)^2}\right), \quad (2.11)$$

где y^{ct} — допустимая статическая ошибка, $t_{\max}$ — требуемая длительность переходного процесса.

Для робота SAMURAI разумные параметры:

$$\mathbf{Q} = \text{diag}(10, 10, 5, 1, 1), \quad \mathbf{R} = \text{diag}(1, 1). \quad (2.12)$$

Большие веса на p_x, p_y, θ означают приоритет точности позиции и курса; малые веса на v, ω — допускаем заметные отклонения скоростей. $\mathbf{R} = \mathbf{I}$ — штрафует затраты управления равномерно.

Примечание

«Сделать поведение робота ошибочным» в терминах ЛКР означает:

- Заменить $\mathbf{Q}$ на отрицательно определённую — алгоритм не сходится или даст нестабильный регулятор;
- Поменять знак $\mathbf{R}$ или сделать $\mathbf{R}$ очень малой — получим избыточно агрессивное управление с насыщением и автоколебаниями;
- Намеренно использовать желаемые полюса в правой полуплоскости $\lambda_i^{\text{уст}} > 0$ для модального регулятора — замкнутая система будет неустойчива.

Эти эксперименты безопасны в симуляторе, но опасны на реальном железе — проводить только с поднятыми колёсами.

2.4 Регулирование с ненулевой уставкой

ЛКР стабилизирует систему в нулевой точке. Для отслеживания заданной траектории $\mathbf{x}_0 \neq \mathbf{0}$ (например, цели Nav2) требуется дополнительное управляющее воздействие (п. 3.2 учебника, формула 3.7):

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t) + \mathbf{u}_{\text{уст}}(t), \quad \mathbf{u}_{\text{уст}} = -(\mathbf{C}\mathbf{A}_z^{-1}\mathbf{B})^{-1}\mathbf{y}_{\text{уст}}(t), \quad (2.13)$$

где $\mathbf{A}_z = \mathbf{A} - \mathbf{BK}$ — матрица замкнутой системы. Это **прямая связь по уставке** (feedforward), компенсирующая статическую ошибку при ступенчатом изменении задания.

Для робота уставка $\mathbf{y}_{\text{уст}} = (p_x^*, p_y^*, \theta^*, v^*, 0)^T$ формируется из заданной точки и желаемой скорости подхода.

2.5 Наблюдатель состояния (Луенбергер)

Регулятор (2.2) использует полный вектор $\mathbf{x}(t)$, но реальные датчики дают только $\mathbf{y} = \mathbf{C}\mathbf{x}$. Согласно п. 3.4.1 учебника, состояние восстанавливается полноразмерным наблюдателем:

Уравнение наблюдателя (Luenberger)

$$\dot{\hat{\mathbf{x}}}(t) = \mathbf{A}\hat{\mathbf{x}} + \mathbf{B}\mathbf{u} + \mathbf{L}(\mathbf{y} - \mathbf{C}\hat{\mathbf{x}}). \quad (2.14)$$

Ошибка восстановления $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$ удовлетворяет:

$$\dot{\mathbf{e}} = (\mathbf{A} - \mathbf{LC})\mathbf{e}. \quad (2.15)$$

Матрица $\mathbf{L}$ выбирается так, чтобы $(\mathbf{A} - \mathbf{LC})$ имела заданные собственные значения, обеспечивающие сходимость $\mathbf{e}(t) \rightarrow 0$ существенно быстрее динамики регулятора.

Стандартная рекомендация (п. 3.4.2 учебника): полюса наблюдателя в $5 \dots 10$ раз быстрее полюсов регулятора:

$$\tilde{\lambda}_i^{\text{набл}} = 5 \cdot \lambda_i^{\text{уст}}. \quad (2.16)$$

2.5.1 Принцип разделения

При совместном использовании регулятора и наблюдателя замкнутая система имеет блочную матрицу (формула 3.23 учебника):

$$\mathbf{A}_c = \begin{pmatrix} \mathbf{A} + \mathbf{BK} & -\mathbf{BK} \\ \mathbf{0} & \mathbf{A} - \mathbf{LC} \end{pmatrix}. \quad (2.17)$$

Спектр $\sigma(\mathbf{A}_c) = \sigma(\mathbf{A} + \mathbf{BK}) \cup \sigma(\mathbf{A} - \mathbf{LC})$ — полюса регулятора и наблюдателя независимы (**принцип разделения**). Это позволяет проектировать $\mathbf{K}$ и $\mathbf{L}$ **независимо**.

2.5.2 Условие наблюдаемости

Существование $\mathbf{L}$, обеспечивающего любые желаемые собственные значения, требует полной наблюдаемости (п. 3.4.1):

$$\text{rank } \mathbf{S}_n = n, \quad \mathbf{S}_n = \begin{pmatrix} \mathbf{C}^\top & \mathbf{A}^\top \mathbf{C}^\top & \dots & (\mathbf{A}^\top)^{n-1} \mathbf{C}^\top \end{pmatrix}. \quad (2.18)$$

При $\mathbf{C} = \mathbf{I}$ (как в нашей модели — все 5 координат наблюдаемы) это условие выполнено тривиально.

2.6 Численный пример для робота

Применим теорию к модели (1.8) при $v_0 = 0.2$ м/с, $T_s = 0.05$ с. Дискретизация ZOH даёт:

$$\mathbf{A}_d \approx \begin{pmatrix} 1 & 0 & 0 & 0.0488 & 0 \\ 0 & 1 & 0.01 & 0 & 0.000244 \\ 0 & 0 & 1 & 0 & 0.0488 \\ 0 & 0 & 0 & 0.717 & 0 \\ 0 & 0 & 0 & 0 & 0.607 \end{pmatrix}.$$

Решение DARE с $\mathbf{Q} = \text{diag}(10, 10, 5, 1, 1)$, $\mathbf{R} = \mathbf{I}_2$ (вычисление в `matlab/design_lqr.m`, см. главу 4) даёт регулятор:

$$\mathbf{K}_{\text{LQR}} \approx \begin{pmatrix} 2.84 & 0 & 0 & 1.21 & 0 \\ 0 & 0.95 & 1.62 & 0 & 0.74 \end{pmatrix}. \quad (2.19)$$

Структура $\mathbf{K}$ отражает связи: u_v зависит от p_x и v (продольный канал), u_ω — от p_y, θ, ω (боковой канал, через рыскание).

Собственные значения замкнутой непрерывной системы $\mathbf{A} - \mathbf{BK}$ лежат в левой полуплоскости (асимптотическая устойчивость), модули собственных значений $|\lambda_i|$ соответствуют постоянным времени $0.3 \dots 1$ с, что обеспечивает время регулирования ~ 1.5 с.

Примечание

ЛКР даёт автоматический компромисс «точность/затраты» через выбор $\mathbf{Q}, \mathbf{R}$. Для расширения на **ограниченные** управления (физические лимиты: $|u_v| \leq 0,3$ м/с, $|u_\omega| \leq 2$ рад/с) переходим к МРС — следующая глава.

Chapter 3

Прогнозирующее управление (МРС)

ЛКР, рассмотренный в главе 2, даёт оптимальный регулятор **без учёта ограничений** на управление. Реальный робот SAMURAI имеет жёсткие физические лимиты:

$$|u_v| \leq u_v^{\max} = 0,3 \text{ м/с}, \quad |u_\omega| \leq u_\omega^{\max} = 2,0 \text{ рад/с}. \quad (3.1)$$

Если ЛКР рассчитывает $u_v = 0,5 \text{ м/с}$, актуатор насытит на $0,3$ — возникает анти-винд-ап эффект, и оптимальность теряется.

Model Predictive Control (МРС) решает эту задачу: на каждом шаге решается *оптимизационная задача* с явным учётом ограничений на горизонте N шагов вперёд.

3.1 Идея метода

В момент времени $t = kT_s$:

1. Измеряется текущее состояние $\mathbf{x}_k$ (или его оценка $\hat{\mathbf{x}}_k$ от наблюдателя).
2. Решается задача оптимизации: найти последовательность $\mathbf{u}_k, \mathbf{u}_{k+1}, \dots, \mathbf{u}_{k+N-1}$, минимизирующую функционал на горизонте.
3. Применяется только *первый* элемент $\mathbf{u}_k$.
4. На следующем шаге $k+1$ горизонт сдвигается — **принцип сдвигаемого горизонта** (receding horizon).

Примечание

МРС можно рассматривать как ЛКР с конечным горизонтом плюс ограничения. При $N \rightarrow \infty$ и без ограничений МРС сводится к ЛКР с тем же матрицей $\mathbf{K}$.

3.2 Математическая постановка

Для дискретной модели (1.12):

Задача MPC на горизонте N

$$\min_{\{\mathbf{u}_i\}_{i=k}^{k+N-1}} J = \sum_{i=k}^{k+N-1} (\mathbf{x}_i^\top \mathbf{Q} \mathbf{x}_i + \mathbf{u}_i^\top \mathbf{R} \mathbf{u}_i) + \mathbf{x}_{k+N}^\top \mathbf{P}_f \mathbf{x}_{k+N} \quad (3.2)$$

$$\text{при условиях: } \mathbf{x}_{i+1} = \mathbf{A}_d \mathbf{x}_i + \mathbf{B}_d \mathbf{u}_i, \quad i = k, \dots, k+N-1, \quad (3.3)$$

$$\mathbf{u}_{\min} \leq \mathbf{u}_i \leq \mathbf{u}_{\max}, \quad (3.4)$$

$$\mathbf{x}_{\min} \leq \mathbf{x}_i \leq \mathbf{x}_{\max}. \quad (3.5)$$

Здесь $\mathbf{P}_f$ — терминальный штраф (terminal cost), обычно выбирается как решение уравнения Риккати $\mathbf{P}_\infty$ — это гарантирует устойчивость замкнутой системы при N достаточно большом.

3.3 Сведение к QP

Задача (3.2)–(3.5) — квадратичная программа (QP) по переменной $\mathbf{U} = (\mathbf{u}_k^\top, \dots, \mathbf{u}_{k+N-1}^\top)^\top \in \mathbb{R}^{rN}$.

3.3.1 Развёртка динамики

Подставив (3.3) итеративно, выразим все будущие состояния через текущее $\mathbf{x}_k$ и $\mathbf{U}$:

$$\begin{pmatrix} \mathbf{x}_{k+1} \\ \mathbf{x}_{k+2} \\ \vdots \\ \mathbf{x}_{k+N} \end{pmatrix} = \underbrace{\begin{pmatrix} \mathbf{A}_d \\ \mathbf{A}_d^2 \\ \vdots \\ \mathbf{A}_d^N \end{pmatrix}}_{\Phi} \mathbf{x}_k + \underbrace{\begin{pmatrix} \mathbf{B}_d & 0 & \dots & 0 \\ \mathbf{A}_d \mathbf{B}_d & \mathbf{B}_d & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_d^{N-1} \mathbf{B}_d & \mathbf{A}_d^{N-2} \mathbf{B}_d & \dots & \mathbf{B}_d \end{pmatrix}}_{\Gamma} \mathbf{U}. \quad (3.6)$$

3.3.2 QP в стандартной форме

Подставим (3.6) в (3.2), получим:

Стандартная QP-задача MPC

$$\min_{\mathbf{U}} \frac{1}{2} \mathbf{U}^\top \mathbf{H} \mathbf{U} + \mathbf{f}^\top(\mathbf{x}_k) \mathbf{U} \quad \text{при} \quad \mathbf{G} \mathbf{U} \leq \mathbf{g}(\mathbf{x}_k), \quad (3.7)$$

где

$$\mathbf{H} = 2(\Gamma^\top \bar{\mathbf{Q}} \Gamma + \bar{\mathbf{R}}), \quad \mathbf{f}(\mathbf{x}_k) = 2\Gamma^\top \bar{\mathbf{Q}} \Phi \mathbf{x}_k,$$

$$\bar{\mathbf{Q}} = \text{blkdiag}(\mathbf{Q}, \dots, \mathbf{Q}, \mathbf{P}_f), \quad \bar{\mathbf{R}} = \text{blkdiag}(\mathbf{R}, \dots, \mathbf{R}).$$

Матрица $\mathbf{H}$ постоянна (не зависит от $\mathbf{x}_k$) и может быть вычислена offline. Вектор $\mathbf{f}$ и правая часть $\mathbf{g}$ ограничений пересчитываются на каждом шаге.

3.4 Алгоритм решения

Для робота SAMURAI используется простой и надёжный подход:

3.4.1 Без ограничений: явное решение

Если ограничения (3.4) неактивны (управление лежит внутри допустимой области), QP (3.7) имеет аналитическое решение:

$$\mathbf{U}^* = -\mathbf{H}^{-1} \mathbf{f}(\mathbf{x}_k) = -\mathbf{K}_{\text{MPC}} \mathbf{x}_k, \quad (3.8)$$

где $\mathbf{K}_{\text{MPC}} \in \mathbb{R}^{r \times n}$ — *статический* коэффициент, вычисляемый один раз. Применяется первая строка размера $r \times n$ (первое управление $\mathbf{u}_k$).

Это ровно та же $\mathbf{K}$, что и у ЛКР, при $N \rightarrow \infty$ и $\mathbf{P}_f = \mathbf{P}_\infty$. Для $N = 10 \dots 30$ результат близок к ЛКР, но позволяет ввести ограничения.

3.4.2 С ограничениями: QP-солвер

При активных ограничениях задача (3.7) решается итеративно. В нашей реализации (см. главу 4) используется простой **проекционный метод**: после явного решения (3.8) результат проецируется на бокс $[\mathbf{u}_{\min}, \mathbf{u}_{\max}]$:

$$\mathbf{u}_k^{\text{clip}} = \max(\mathbf{u}_{\min}, \min(\mathbf{u}_{\max}, \mathbf{u}_k^*)). \quad (3.9)$$

Примечание

Это **не оптимальное** решение QP с ограничениями (для оптимальности нужен active-set или interior-point), но даёт корректное и устойчивое поведение для робота. Полноценный QP-солвер (cvxpy, qpOASES, scipy.optimize.minimize) можно подключить опцией control.mpc.solver: qp в конфиге.

3.4.3 Терминальный штраф $\mathbf{P}_f$

Согласно теореме об устойчивости MPC (Mayne et al., 2000), терминальный штраф $\mathbf{P}_f$, выбранный как решение уравнения Риккати замкнутой системы $\mathbf{A}_d - \mathbf{B}_d \mathbf{K}_\infty$, гарантирует устойчивость MPC при любом $N \geq 1$. Поэтому в matlab/design_mpc.m сначала решается DARE для получения $\mathbf{P}_f = \mathbf{P}_\infty$, затем формируется QP.

3.5 Параметры MPC для SAMURAI

Параметр	Значение	Комментарий
T_s	0,05 с	период цикла motor_node (20 Гц)
N	10	горизонт 0,5 с (соизмерим с τ_v)
$\mathbf{Q}$	diag(10, 10, 5, 1, 1)	точность позиции/курса
$\mathbf{R}$	diag(1, 1)	умеренный штраф управления
$\mathbf{P}_f$	решение DARE	гарантия устойчивости
$\mathbf{u}_{\min}$	(−0,3, −2,0)	физические лимиты робота
$\mathbf{u}_{\max}$	(+0,3, +2,0)	— simulator.max_*

При $N = 10$, $n = 5$, $r = 2$ задача QP имеет 20 переменных, 40 ограничений-неравенств. Время решения на Raspberry Pi 4 — менее 1 мс (запас порядка для 50 мс цикла).

3.6 Подбор поведения и «ошибочные» режимы

Параметры МРС вынесены в `config.yaml` и могут быть изменены без перекомпиляции. Это позволяет:

1. Корректные поведения

- **Точный медленный режим:** $Q \uparrow$, $R \uparrow$, $u_{\max} \downarrow$ — робот ездит медленно, но точно.
- **Агрессивный режим:** $Q_{p_x, p_y} \uparrow\uparrow$, $R \downarrow$ — быстрая отработка, рывки.
- **Длинный горизонт:** $N = 30$ — видит дальше, плавнее огибает препятствия (но дороже по CPU).

2. «Ошибочные» поведения для отладки

- **Отрицательный штраф:** $Q = -I$ — QP станет невыпуклой, регулятор будет «гнать» в неустойчивость.
- **Завышенные лимиты:** $u_{\max} = 100$ — ЛКР перестанет насыщаться, проявится колебание из-за неточностей модели.
- **Полюса в правой полуплоскости** (модальный режим): $\lambda^{\text{уст}} = +1$ — замкнутая система неустойчива, робот «убегает».
- **Зануление обратной связи по позиции:** $Q_{1,1} = Q_{2,2} = 0$ — робот игнорирует координаты p_x, p_y , движется только по внутренним скоростям.

Внимание

Все «ошибочные» режимы безопасны в симуляторе (`./samurai.sh sim`), но опасны на реальном железе. Для физических испытаний:

- Поднимайте робота на подставке — колёса не должны касаться пола;
- Включите аппаратный аварийный стоп (`config.motor.collision_guard_stop_m`);
- Не превышайте $u_{\max} = 0,5$ м/с.

Chapter 4

Реализация в коде робота

В этой главе описывается **полный конвейер**: MATLAB (offline) $\rightarrow$ YAML/JSON $\rightarrow$ Python (online) $\rightarrow$ motor_node реального робота. Все коэффициенты выносятся в config.yaml, что позволяет менять поведение робота без перекомпиляции.

4.1 Архитектура



Шаги работы:

1. Инженер запускает main.m в MATLAB — получает $\mathbf{A}, \mathbf{B}, \mathbf{K}, \mathbf{L}, \mathbf{P}_f$;
2. Скрипт автоматически экспортирует матрицы в config.yaml раздел control;
3. Робот при старте читает control и инициализирует регулятор;
4. На каждом цикле motor_node вызывает $\mathbf{u}_k = \text{controller.step}(\mathbf{x}_k, \mathbf{x}_k^{\text{ref}})$;
5. Если control.mode = off, действует старая логика прямой трансляции cmd_vel.

4.2 Конфигурация

Все коэффициенты вынесены в config.yaml, раздел control:

config.yaml — control:

```

1 control:
2   mode: "off"          # off | lqr | mpc | modal — режим регулято
   pa
3
4   plant:               # параметры объекта (физика)
5     v0:                0.20    # м/с — опорная скорость линеаризации
6     tau_v:             0.15    # с — постоянная мотора (linear)
7     tau_w:             0.10    # с — постоянная мотора (angular)
8     Ts:                0.05    # с — период дискретизации (= 1 / 20 Hz)
9
10  weights:             # критерий качества
11    Q_diag: [10, 10, 5, 1, 1]  # x,y,theta,v,omega
12    R_diag: [1, 1]          # u_v, u_omega
13
14  limits:              # физические ограничения управления
15    u_min: [-0.30, -2.0]
16    u_max: [+0.30, +2.0]
17
18  mpc:                 # параметры MPC
19    horizon_N: 10         # горизонт прогноза (шагов)
20    solver: "clip"       # clip | qr (qr требует scipy/cvxpy)
21
22  modal:               # параметры модального регулятора
23    poles_continuous: [-2, -3, -4, -8, -10] # желаемые
24    observer_speedup: 5.0 # во сколько раз быстрее регулятора
25
26  # Матрицы рассчитаны MATLAB-скриптом, обновляются автоматически.
27  # Для ручной правки: установите override: true и поменяйте значе
   ния.
28  matrices:
29    override: false
30    A: null # 5x5 — заполнится из MATLAB
31    B: null # 5x2
32    K: null # 2x5 — главный регулятор
33    L: null # 5x5 — наблюдатель (если включен)
34    Pf: null # 5x5 — терминальный штраф MPC

```

4.3 Python-модуль управления

В пакете `pi_nodes/control/` добавлены три новых модуля:

Файл	Назначение
<code>state_space_model.py</code>	Дискретная модель $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$
<code>lqr_controller.py</code>	ЛКР: $\mathbf{u}_k = -\mathbf{K} \mathbf{x}_k$
<code>mpc_controller.py</code>	МРС: явное решение + проекция на бокс

4.3.1 Загрузка матриц

pi_nodes/control/state_space_model.py

```

1 import numpy as np
2 from config_loader import cfg
3
4 class StateSpaceModel:
5     """Discrete state-space model  $x[k+1] = A_d x[k] + B_d u[k]$ ."""
6     def __init__(self, Ad=None, Bd=None):
7         if Ad is None:
8             Ad = np.array(cfg('control.matrices.A'), dtype=float)
9         if Bd is None:
10            Bd = np.array(cfg('control.matrices.B'), dtype=float)
11        self.Ad, self.Bd = Ad, Bd
12        self.n, self.r = self.Bd.shape
13
14    def step(self, x, u):
15        return self.Ad @ x + self.Bd @ u

```

4.3.2 Регулятор LQR

pi_nodes/control/lqr_controller.py

```

1 import numpy as np
2 from config_loader import cfg
3
4 class LQRController:
5     def __init__(self, K=None, u_min=None, u_max=None):
6         self.K = np.array(K if K is not None
7                             else cfg('control.matrices.K'))
8         self.u_min = np.array(u_min if u_min is not None
9                                 else cfg('control.limits.u_min'))
10        self.u_max = np.array(u_max if u_max is not None
11                                else cfg('control.limits.u_max'))
12
13    def step(self, x, x_ref=None):
14        dx = x - x_ref if x_ref is not None else x
15        u = -self.K @ dx
16        return np.clip(u, self.u_min, self.u_max)

```

4.3.3 MPC: явное решение + проекция

pi_nodes/control/mpc_controller.py (упрощённо)

```

1 class MPCController:
2     """MPC с явным решением + проекцией на бокс ограничений."""
3
4     def __init__(self, Ad, Bd, Q, R, Pf, N, u_min, u_max):
5         self.N = N
6         self._build_qp_matrices(Ad, Bd, Q, R, Pf)
7         self.u_min, self.u_max = u_min, u_max
8         self.r = Bd.shape[1]
9
10    def _build_qp_matrices(self, Ad, Bd, Q, R, Pf):
11        n, r = Bd.shape
12        N = self.N
13        # Phi (n*N x n) и Gamma (n*N x r*N) -- развёртка динамики
14        Phi = np.vstack([np.linalg.matrix_power(Ad, i+1) for i
15                        in range(N)])
16        Gamma = np.zeros((n*N, r*N))
17        for i in range(N):
18            for j in range(i+1):
19                block = np.linalg.matrix_power(Ad, i-j) @ Bd
20                Gamma[i*n:(i+1)*n, j*r:(j+1)*r] = block
21
22        Q_bar = np.kron(np.eye(N-1), Q)
23        Q_bar = block_diag_pad(Q_bar, Pf) # терминальны
24        # штраф
25        R_bar = np.kron(np.eye(N), R)
26
27        H_full = 2*(Gamma.T @ Q_bar @ Gamma + R_bar)
28        f_mat = 2*Gamma.T @ Q_bar @ Phi
29
30        # Только первое управление: K_first = (H^-1 f)[:r, :]
31        K_first = -np.linalg.solve(H_full, f_mat)[:r, :]
32        self.K_first = K_first
33
34    def step(self, x, x_ref=None):
35        dx = x - x_ref if x_ref is not None else x
36        u = self.K_first @ dx
37        return np.clip(u, self.u_min, self.u_max)

```

4.4 Тестирование

Для каждого модуля написан unit-тест в tests/.

4.4.1 test_state_space_model.py

Проверяет:

- $\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d \mathbf{u}_k$ — корректность дискретного шага;
- Сходимость к точке равновесия при $\mathbf{u} = \mathbf{0}$ (для устойчивого $\mathbf{A}_d$).

4.4.2 test_lqr_controller.py

Проверяет:

- Замкнутая система $\mathbf{A}_d - \mathbf{B}_d \mathbf{K}$ устойчива ($|\lambda_i| < 1$);
- Управление лежит в пределах $[\mathbf{u}_{\min}, \mathbf{u}_{\max}]$;
- При $\mathbf{x} = \mathbf{0}$ управление равно нулю.

4.4.3 test_mpc_controller.py

Проверяет:

- Без ограничений MPC совпадает с ЛКР для горизонта $N \geq 20$;
- При активных ограничениях управление лежит на границе бокса;
- Замкнутая симуляция сходится к нулю.

4.5 Примеры использования

4.5.1 Запуск симуляции с MPC

Изменение режима регулятора

```
1 # 1. В config.yaml:  
2 control:  
3   mode: mpc  
4  
5 # 2. Запустить симулятор:  
6 ./samurai.sh sim  
7  
8 # 3. Открыть dashboard, отправить goal_pose
```


4.5.2 Расчёт нового регулятора в MATLAB

matlab/main.m — сценарий

```

1 % Загрузить параметры объекта
2 params = samurai_params();
3 % Линеаризовать
4 [A, B, C, D] = linearize_samurai(params);
5 % Дискретизировать
6 [Ad, Bd] = discretize_samurai(A, B, params.Ts);
7 % Синтез ЛКР
8 [K_lqr, P] = design_lqr(Ad, Bd, params.Q, params.R);
9 % Синтез MPC
10 mpc = design_mpc(Ad, Bd, params.Q, params.R, P, params.N)
11 ;
12 % Экспорт в config.yaml
13 export_to_yaml(Ad, Bd, K_lqr, P, mpc, '../config.yaml');
14 % Симуляция
15 simulate_closed_loop(Ad, Bd, K_lqr);

```

4.5.3 «Ошибочный» режим для демонстрации

Чтобы показать студентам **неустойчивую систему**, достаточно поставить в config.yaml:

Демонстрация неустойчивости

```

1 control:
2   mode: modal
3   modal:
4     poles_continuous: [+1.0, +0.5, -1, -1, -1] # 2 положительных!

```

При запуске симуляции робот будет «убегать» по экспоненте, так как замкнутая система имеет полюса в правой полуплоскости.

4.6 Заключение

Резюме созданной системы:

1. Построена 5-мерная нелинейная ОДУ-модель робота (глава 1);
2. Модель линеаризована вокруг режима прямолинейного движения и дискретизирована с $T_s = 0,05$ с;
3. Синтезированы 3 регулятора: модальный (pole placement), ЛКР (Risk-метод), MPC с горизонтом $N = 10$ и проекцией;
4. Полноразмерный наблюдатель восстанавливает все 5 координат состояния по доступным измерениям;
5. Расчёт выполняется offline в MATLAB (matlab/, 6 файлов), результат экспортируется в config.yaml;

6. Python-реализация (`pi_nodes/control/`, 3 файла) читает конфиг при старте и применяет регулятор в цикле `motor_node` (20 Гц);
7. Покрыто unit-тестами (`tests/test_*_controller.py`, 3 файла);
8. Все параметры $\mathbf{Q}$, $\mathbf{R}$, N , $\mathbf{u}_{\min, \max}$ — редактируемы из конфига без перекомпиляции, что позволяет как улучшать поведение робота, так и намеренно вводить ошибки для отладки и обучения.

Ссылки на файлы проекта:

Файл	Содержимое
<code>matlab/main.m</code>	оркестратор расчёта
<code>matlab/samurai_params.m</code>	параметры объекта
<code>matlab/linearize_samurai.m</code>	матрицы $\mathbf{A}$, $\mathbf{B}$
<code>matlab/discretize_samurai.m</code>	ZOH-дискретизация
<code>matlab/design_lqr.m</code>	решение DARE
<code>matlab/design_mpc.m</code>	МРС-матрицы
<code>matlab/simulate_closed_loop.m</code>	симуляция
<code>matlab/export_to_yaml.m</code>	запись конфига
<code>pi_nodes/control/state_space_model.py</code>	дискретная модель
<code>pi_nodes/control/lqr_controller.py</code>	ЛКР-регулятор
<code>pi_nodes/control/mpc_controller.py</code>	МРС-регулятор
<code>config.yaml</code>	раздел <code>control</code> :
<code>tests/test_state_space_model.py</code>	тест модели
<code>tests/test_lqr_controller.py</code>	тест ЛКР
<code>tests/test_mpc_controller.py</code>	тест МРС